Machine Learning-Based Fraud and Anomaly Detection for Anti-Money Laundering Systems

Inderpreet Doad, Musa Khalid, Mouiz Majid, Shivam Patel
Department of Computer Science
Western University
London, Ontario, Canada
{idoad, mmajid7, mkhal, spate739}@uwo.ca

***Abstract*—Anti-money laundering systems operate in a setting where fraudulent transactions are rare, expensive to miss, and often difficult to separate from legitimate but unusual activity. In this work, we build a machine learning pipeline for fraud and anomaly detection using the PaySim dataset, a large synthetic dataset that simulates mobile money transactions. The pipeline includes data cleaning, feature engineering, stratified train, validation, and test splitting, preprocessing, class imbalance handling, supervised and unsupervised model training, threshold tuning, and a data leakage experiment. We evaluate Logistic Regression, Random Forest, XGBoost, Isolation Forest, One-Class SVM, and Local Outlier Factor using precision, recall, F1 score, ROC-AUC, and PR-AUC instead of relying on accuracy alone. The results show that XGBoost gives the strongest fraud detection performance, while unsupervised anomaly detection methods perform much worse than supervised methods when labels are available. We also show that balance-related leakage features can greatly inflate performance, which highlights the importance of careful feature selection. Overall, the results show that effective AML fraud detection depends on reliable features, appropriate evaluation under severe class imbalance, and an experimental setup that reflects real deployment conditions.**

***Index Terms*—anti-money laundering, fraud detection, anomaly detection, class imbalance, data leakage, XGBoost, Random Forest, PaySim**

I. Introduction

Financial fraud and money laundering remain major challenges for modern financial institutions. Banks and payment platforms process millions of transactions every day, and suspicious behaviour is often hidden among normal activity. Missing even a small amount of fraud can lead to financial losses, regulatory penalties, and reputational harm. At the same time, flagging too many legitimate transactions creates extra work for investigators and increases operating costs.

Fraud detection in this setting is not a typical binary classification problem. In the PaySim dataset used in this study, only about 0.13% of all transactions are fraudulent. A model that predicts every transaction as legitimate would still achieve about 99.87% accuracy. Although that sounds impressive, such a model would fail to detect any fraud. For this reason, accuracy is not a useful measure of performance here.

Instead, we focus on metrics that better reflect the real goal of an AML system. Precision, recall, F1 score, ROC-AUC, and PR-AUC provide a much clearer picture of whether a model can detect rare fraudulent behaviour without overwhelming the system with false alarms.

This study has four main goals:

1) Build a complete and modular fraud detection pipeline from raw transaction data to final evaluation.
2) Compare supervised learning with unsupervised anomaly detection.
3) Address extreme class imbalance through class weighting and threshold tuning.
4) Show how data leakage can artificially improve performance and give a misleading picture of model quality.

The last goal is especially important. A model can look very strong during offline evaluation while actually learning patterns that would not be available in practice. One of the main lessons from this work is that reliable AML modelling depends not only on the algorithm, but also on careful feature design and realistic evaluation.

II. Problem Context

Fraud detection in AML is fundamentally a tradeoff. Institutions want to catch as much fraud as possible, but they also need to limit false alarms. A system that labels every transaction as fraud would identify all true fraud cases, but it would be unusable in practice. On the other hand, a system that only flags the most obvious cases might have high precision but still miss a large share of fraud.

This is why the main performance metrics need to be interpreted carefully:

- **Precision** asks: when the model flags a transaction, how often is it actually fraud?
- **Recall** asks: of all fraudulent transactions, how many does the model catch?
- **F1 score** combines precision and recall into one metric and penalizes models that do well on only one of them.

A model with high recall and low precision may detect many fraudulent cases, but it also wastes investigators' time with too many false positives. A model with high precision and low recall raises more trustworthy alerts, but lets too much fraud pass through. The F1 score is useful because it captures the balance between these two objectives.

III. Dataset

The dataset used in this study is **PaySim**, a synthetic financial transaction dataset generated to reflect real mobile money transaction behaviour. It is widely used in fraud detection research because real banking data are difficult to access due to privacy, security, and regulatory constraints.

The dataset contains:

- approximately 6.3 million transactions,
- 11 original features,
- 5 transaction types,
- a fraud rate of approximately 0.13%.

The original columns include the transaction step, transaction type, amount, account identifiers, sender and receiver balances before and after the transaction, and fraud labels. Although the dataset is synthetic, it still reflects realistic transaction flow and severe class imbalance, which makes it a useful benchmark for AML-related modelling.

Fig. 1 shows the class distribution. The non-fraud class makes up almost the entire dataset, while fraudulent transactions represent only a very small fraction. This is exactly why an accuracy-based evaluation would be misleading.

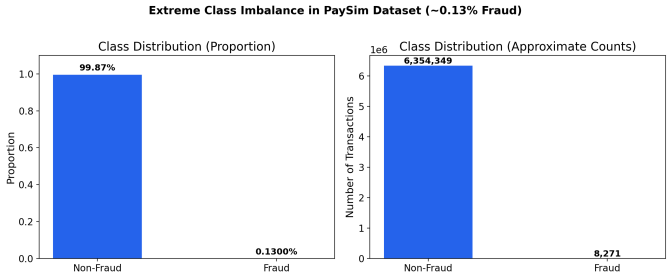

Fig. 1: Extreme class imbalance in the PaySim dataset. Fraud accounts for only about 0.13% of all transactions.

IV. System Design and Repository Structure

One strength of this work is the modular structure of the codebase. Instead of putting the entire pipeline into one large script, the project separates each stage into its own file. This makes the system easier to understand, test, and extend.

At the highest level, `main.py` runs the full workflow:

1) load the dataset,
2) clean the data,
3) engineer features,
4) build feature sets,
5) split the data,
6) preprocess inputs,
7) train models,
8) evaluate results,
9) run the leakage experiment,
10) save figures, metrics, and artifacts.

The main modules are:

- `src/data_preprocessing.py`: loads data, cleans it, splits it, and builds preprocessors.
- `src/feature_engineering.py`: creates the derived features used by the models.
- `src/models.py`: defines model configurations.
- `src/train.py`: handles model training.
- `src/evaluate.py`: computes metrics, tunes thresholds, and performs the leakage comparison.
- `src/utils.py`: generates plots and saves output files.

The repository also includes utility scripts:

- `scripts/demo_score_new_data.py` for scoring new transactions,
- `scripts/make_demo_samples.py` for generating synthetic demo inputs,
- `scripts/generate_presentation_assets.py` for creating presentation-ready charts and summary files.

This structure makes the project easier to maintain. For example, a new engineered feature can be added in `feature_engineering.py` without changing the rest of the workflow. In the same way, new metrics or plots can be added without rewriting the training code.

V. Methodology

A. Data Cleaning and Preprocessing

The pipeline starts by loading the raw CSV file and preparing it for model training. Duplicate rows are removed when needed, data types are standardized, and variables are separated into numerical and categorical groups.

Preprocessing is done with `ColumnTransformer`. Numeric features are imputed and, when appropriate, scaled. Categorical features are imputed and one-hot encoded. We use separate scaled and unscaled preprocessing pipelines because some models are sensitive to feature scale, while tree-based models are not.

This matters because preprocessing is part of the final model pipeline. The same transformations used during training must also be applied during inference so that new data are processed in a consistent way.

B. Feature Engineering

The raw PaySim variables do not capture all of the behavioural patterns that may be linked to fraud. To improve this, we create additional features designed to make suspicious activity easier to detect. These include:

- time-based features such as hour of day and day of month,
- amount-based features such as log-transformed amount and high-amount flags,
- balance-change and ratio features,
- merchant indicators,
- aggregated account behaviour such as transaction counts and mean amounts.

This step is important because fraud is usually not identified from one raw variable alone. In many cases, suspicious behaviour depends on timing, transaction type, size, and surrounding account activity rather than a single isolated number.

C. Train, Validation, and Test Split

The dataset is split into three parts:

- training set: 64%,
- validation set: 16%,
- test set: 20%.

The split is stratified so that each partition keeps roughly the same small fraud proportion. This is important because the model should be trained and evaluated under class conditions that match the full dataset.

The training set is used to fit the models, the validation set is used for threshold tuning, and the test set is reserved for the final evaluation.

D. Baseline Model

Before testing more advanced methods, we evaluate a baseline model that always predicts the majority class, which is non-fraud. The purpose of this baseline is to show how misleading accuracy can be under extreme class imbalance.

This comparison makes it clear that high accuracy alone says very little about real fraud detection ability.

E. Handling Class Imbalance

Because fraud is extremely rare, the models need to place more importance on fraud examples during training. Otherwise, the learning process would be dominated by the much larger non-fraud class.

We handle this using:

- `class_weight='balanced'` for Logistic Regression and Random Forest,
- `scale_pos_weight` for XGBoost based on the class ratio.

In practice, this means that misclassifying a fraud example is treated as more costly than misclassifying a typical legitimate transaction.

F. Supervised Models

We evaluate three supervised models:

- **Logistic Regression**: a simple and interpretable linear baseline that is limited when fraud patterns are nonlinear.
- **Random Forest**: an ensemble of decision trees that can model more complex interactions and is robust to noisy features.
- **XGBoost**: a boosted tree ensemble where each new tree focuses on correcting earlier errors.

Because the dataset includes fraud labels, supervised learning is expected to perform well in this setting.

G. Unsupervised Models

We also test three unsupervised anomaly detection models:

- **Isolation Forest**,
- **One-Class SVM**,
- **Local Outlier Factor**.

These models are trained only on normal transactions and try to identify observations that look unusual. This can be useful when labels are unavailable. However, unusual activity is not always fraudulent, especially in financial systems where rare but legitimate transactions are common.

H. Threshold Tuning

The models produce probabilities or anomaly scores rather than direct binary decisions. A threshold is therefore needed to convert those outputs into final predictions.

Instead of using the default threshold of 0.5, we tune the threshold on the validation set to maximize F1 score. This is especially useful in rare-event classification, where the default threshold may not produce the best balance between precision and recall.

I. Leakage Experiment

One of the most important parts of the methodology is the leakage comparison experiment. We train the same Random Forest model twice:

- once with a realistic no-leak feature set,
- once with a feature set that includes leakage-sensitive balance variables.

This comparison was included because some balance fields in PaySim are closely tied to the fraud label due to the way fraudulent transactions are simulated. If these columns are included, the model can appear much stronger than it really is.

VI. Results

A. Overall Model Performance

Table I summarizes the main results.

TABLE I: Overall model performance

Model	Precision	Recall	F1	ROC-AUC
XGBoost	0.7052	0.5490	0.6174	0.9821
Random Forest	0.5477	0.4267	0.4796	0.9785
Logistic Regression	0.0671	0.1339	0.0894	0.9337
One-Class SVM	0.0643	0.2690	0.1038	0.7903
Local Outlier Factor	0.0761	0.0773	0.0767	0.8518
Isolation Forest	0.0463	0.0426	0.0444	0.8883

XGBoost is the strongest model overall, followed by Random Forest. Logistic Regression performs much worse, which suggests that the fraud patterns in this dataset are too complex to be captured well by a simple linear boundary.

Fig. 2 shows the same ranking using F1 score.

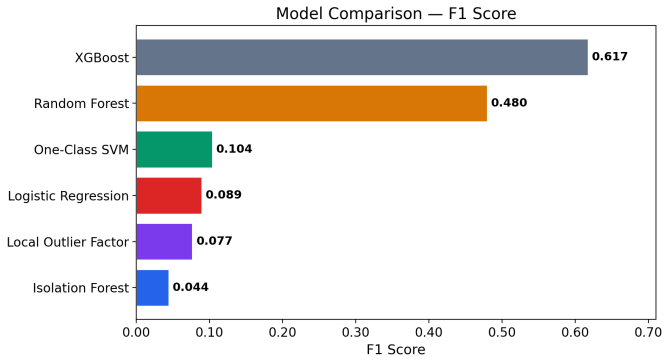

Fig. 2: Model comparison using F1 score. XGBoost achieves the best overall balance of precision and recall.

B. Interpreting Precision, Recall, and F1

The best model, XGBoost, achieves a precision of about 0.705 and a recall of about 0.549. In practical terms, this means:

- when the model flags a transaction as fraud, it is correct about 70% of the time,
- it detects about 55% of all actual fraud cases.

Its F1 score of 0.617 summarizes this tradeoff. The model is far from perfect, but it provides a useful balance between catching fraud and limiting false positives. That is much more informative than reporting accuracy alone.

C. Supervised vs. Unsupervised Learning

Fig. 3 compares the best supervised model with the best unsupervised model.

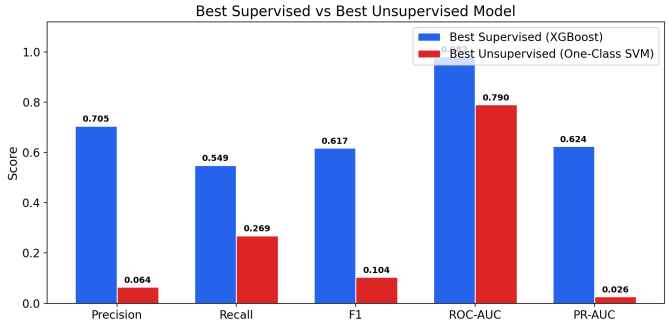

Fig. 3: Best supervised model versus best unsupervised model. Supervised learning is much stronger when labelled data are available.

The difference is clear. Supervised learning performs much better than anomaly detection in this setting. XGBoost gives stronger precision, stronger F1, and much higher PR-AUC than One-Class SVM.

This result is also reasonable from a modelling perspective. Unsupervised methods only learn what normal behaviour looks like and then flag anything that appears unusual. But unusual financial behaviour is not necessarily fraud. Large legitimate transactions, uncommon merchants, and irregular customer actions can all appear anomalous. Supervised models have access to the fraud labels, so they can learn the patterns that actually matter.

D. Feature Importance

To better understand what the models are learning, we examine feature importance for Random Forest and XGBoost.

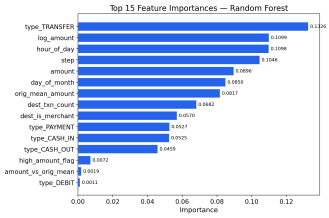

Fig. 4: Top 15 feature importances for Random Forest.

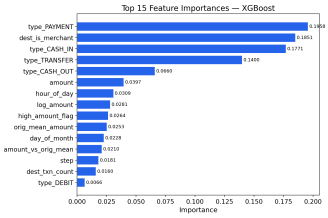

Fig. 5: Top 15 feature importances for XGBoost.

The plots show that transaction type, amount-based variables, time-based features, merchant indicators, and aggregate behavioural statistics all contribute to the predictions. This supports the importance of feature engineering and suggests that fraud detection depends on behavioural context rather than only on raw balance values.

E. Data Leakage Results

The leakage comparison experiment is one of the most important findings in this study. Fig. 6 shows how Random Forest performs with and without leakage-sensitive features.

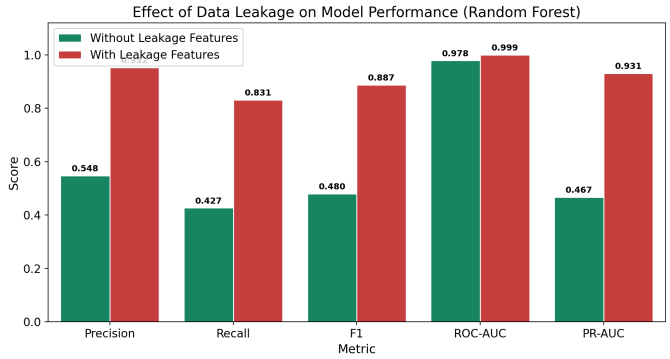

Fig. 6: Effect of data leakage on Random Forest performance. Leakage-sensitive balance features dramatically inflate evaluation metrics.

Without leakage features, Random Forest achieves:

- Precision = 0.548,
- Recall = 0.427,
- F1 = 0.480,
- PR-AUC = 0.467.

With leakage features included, the same model appears to improve sharply:

- Precision = 0.952,
- Recall = 0.831,
- F1 = 0.887,
- PR-AUC = 0.931.

This is not a genuine improvement in model quality. Instead, it shows that the feature set is exposing information that is too closely tied to the label. In a real deployment environment, that information would not be available in the same way, so the model would not reproduce these inflated results. This experiment shows why leakage prevention has to be treated as a core part of fraud detection methodology.

VII. Deployment and Reproducibility

The project is designed not only for model training, but also for practical reuse. After identifying the best supervised model, the full model bundle can be saved as an artifact. This bundle includes:

- the preprocessing pipeline,
- the trained XGBoost classifier,
- the selected threshold,
- the feature metadata needed for inference.

This makes it possible to score new transactions later without retraining the model. The deployment workflow is straightforward:

1) load the saved model artifact,
2) apply the same cleaning and feature engineering steps,
3) transform the data using the saved preprocessor,
4) generate fraud probabilities,
5) apply the tuned threshold,
6) output a fraud or non-fraud decision.

The repository also contains utility scripts for generating synthetic demo transactions and scoring them with the saved model. This supports live demonstrations and makes the system easier to explain to non-technical audiences.

The project is also reproducible. The repository includes:

- a clear file structure,
- a `requirements.txt` file,
- committed metrics and plots,
- a README with setup instructions,
- support for storing the large PaySim CSV through Git LFS.

VIII. Discussion

Several main conclusions follow from the results.

First, **accuracy is not a useful target** in settings with extreme class imbalance. A model can achieve very high accuracy while still being completely ineffective for fraud detection. Precision, recall, F1, and PR-AUC provide a much more meaningful evaluation.

Second, **supervised learning performs better than unsupervised anomaly detection** when labels are available. XGBoost and Random Forest clearly outperform Isolation Forest, One-Class SVM, and Local Outlier Factor.

Third, **feature engineering matters**. Model performance depends not only on algorithm choice, but also on whether raw transaction data are converted into useful behavioural signals.

Fourth, **threshold tuning matters**. The final fraud decision depends not just on the model's predicted probabilities, but also on the cutoff used to turn those probabilities into labels. Validation-based tuning gives a better practical balance between precision and recall.

Fifth, **data leakage can invalidate an otherwise strong project**. The leakage experiment shows that impressive offline results can be misleading if the feature set includes hidden label information. This is one of the most important lessons from the study.

IX. Limitations and Future Work

Although the results are strong, the project still has several limitations.

First, PaySim is a synthetic dataset. It is useful for benchmarking, but real financial systems may involve concept drift, institution-specific behaviour, changing adversarial strategies, and richer customer patterns than a simulator can fully represent.

Second, the current evaluation focuses on standard fraud metrics rather than a fully cost-sensitive decision framework. In practice, financial institutions may want to tune the system based on investigation cost, customer experience, or regulatory risk tolerance.

Third, the current pipeline does not explicitly model graph relationships or long transaction sequences, both of which may be important for money laundering detection.

Future work could include:

- cost-sensitive threshold optimization,
- drift monitoring during deployment,
- API-based real-time scoring,
- graph-based fraud detection,
- temporal or sequential models,
- evaluation on more realistic institutional datasets.

X. Research Foundation and Relevance to This Project

This project is not based only on trial-and-error model building. Its design choices are supported by prior research in fraud detection, anomaly detection, imbalanced classification, and scalable machine learning. The sources used in this report justify the dataset selection, model choices, evaluation metrics, and overall AML framing of the project.

First, the choice of **XGBoost** as one of the main supervised models is supported by Chen and Guestrin's work, which introduces XGBoost as an efficient and scalable tree-boosting system for high-performance predictive modelling [1]. This directly applies to our project because the PaySim dataset is large, highly imbalanced, and contains nonlinear relationships between transaction features and fraud labels. Since simpler linear models may fail to capture these patterns well, XGBoost was included as a strong benchmark for fraud classification.

Second, the use of the **PaySim dataset** is supported by the original PaySim paper by Lopez-Rojas, Elmir, and Axelsson, which explains that the dataset was created to simulate realistic mobile money transactions for fraud detection research [2]. This is important for our project because real AML transaction data are difficult to access due to privacy and regulatory restrictions. PaySim provides a large-scale environment where fraud detection methods can still be tested under conditions similar to real financial transaction systems.

Third, our decision to emphasize **precision, recall, F1 score, and PR-AUC** rather than accuracy is supported by Saito and Rehmsmeier, who show that precision-recall analysis is more informative than ROC analysis when working with highly imbalanced datasets [3]. This directly applies to our

project because fraud makes up only a very small fraction of the PaySim data. In such settings, a model can achieve extremely high accuracy while detecting little or no fraud. Their findings justify our decision to treat accuracy as a weak metric and focus instead on measures that better reflect rare-event detection quality.

Fourth, the inclusion of **unsupervised anomaly detection methods** is supported by Chandola, Banerjee, and Kumar's survey on anomaly detection [4]. Their work explains that anomaly detection methods are useful when abnormal behaviour must be identified without relying entirely on labelled examples. This applies to our project because AML systems often deal with suspicious or rare behaviour that may not always be fully labelled in practice. Their framework supports our decision to compare supervised fraud classification against unsupervised anomaly detection methods.

Fifth, the use of **Isolation Forest** specifically is supported by Liu, Ting, and Zhou, who introduced the method as an efficient anomaly detection algorithm that isolates unusual observations instead of modelling normality in a traditional density-based way [5]. This is relevant to our project because fraudulent transactions can be viewed as rare and unusual observations within a much larger population of legitimate transactions. Including Isolation Forest allowed us to test whether this type of anomaly-based reasoning could detect fraud effectively in comparison with supervised learning.

Finally, the broader framing of this work as a **financial fraud detection problem** is supported by Ngai et al., who review the application of data mining methods in financial fraud detection and show that fraud problems often benefit from classification, pattern recognition, and anomaly detection approaches [6]. This source supports the overall design of our project because it places our work within the larger academic literature on fraud analytics and confirms that combining multiple machine learning strategies is a valid approach for financial risk detection.

Taken together, these references show that the project is grounded in established research rather than arbitrary modelling decisions. The literature supports our use of a fraud simulation dataset, our comparison of supervised and unsupervised methods, our emphasis on imbalance-aware evaluation metrics, and our selection of scalable machine learning models for AML-related fraud detection.

XI. Conclusion

This study developed a complete and modular machine learning pipeline for fraud and anomaly detection in an AML setting using the PaySim dataset. It addressed several core challenges in real-world fraud detection, including severe class imbalance, meaningful evaluation, feature engineering, threshold tuning, and data leakage.

XGBoost was the strongest model, giving the best overall balance of precision, recall, and F1 score. Random Forest also performed well, while the unsupervised methods were much weaker than the supervised approaches. Most importantly, the leakage experiment showed that strong-looking results can be misleading if the feature set is not realistic for deployment.

The main takeaway is that effective fraud detection depends on more than choosing a strong algorithm. It also depends on honest evaluation, careful feature design, and a setup that reflects the real goal of catching rare fraud while limiting false alarms.

Appendix A: Code Repository and Demo Instructions

Click here to access the repository

Presentation Day Commands:

1) **Generate Figures and Tables** `python scripts/generate_presentation_assets.py` Generates all presentation charts, summary tables, and key findings in `presentation_assets/`.
2) **Create Demo Transactions** `python scripts/make_demo_samples.py` Produces `demo_samples.csv` with synthetic transactions.
3) **Run Live Demo Predictions** `python scripts/demo_score_new_data.py` Loads the trained XGBoost model and outputs fraud probabilities and predictions. Results are saved to `demo_predictions.csv`.

Notes:

- Figures and tables are already pre-generated in `presentation_assets/`.
- Demo sample transactions are also pre-created; regeneration is optional.

Appendix B: Codebase Map

This appendix summarizes where to look in the repository when modifying specific parts of the system.

- **Change how data is loaded or cleaned:** `src/data_preprocessing.py`
- **Add or remove engineered features:** `src/feature_engineering.py`
- **Adjust model definitions or hyperparameters:** `src/models.py`
- **Change training logic:** `src/train.py`
- **Change evaluation metrics or threshold tuning:** `src/evaluate.py`
- **Change charts or output formatting:** `src/utils.py`
- **Run the full pipeline:** `main.py`
- **Score new transactions:** `scripts/demo_score_new_data.py`
- **Create demo data:** `scripts/make_demo_samples.py`
- **Generate presentation visuals:** `scripts/generate_presentation_assets.py`

Appendix C: Group Member Contributions

- **Inderpreet Doad:** Project coordination, methodology design, supervised model development, and final submission preparation.
- **Musa Khalid:** Data preprocessing, feature engineering, supervised model development, experimental evaluation.
- **Mouiz Majid:** Unsupervised model development, anomaly detection analysis, and result interpretation.
- **Shivam Patel:** Data visualization , unsupervised model development, presentation and material development.

APPENDIX D: USE OF GENERATIVE AI

Generative AI tools were used only to assist with formatting and minor adjustments, such as improving document structure and fixing hyperlinks. All technical content, analysis, code, and written explanations were developed, reviewed, and finalized by the group members. Final responsibility for the work remains with the group.

REFERENCES

[1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[2] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th European Modeling and Simulation Symposium (EMSS)*, Larnaca, Cyprus, 2016, pp. 249–255. https://www.kaggle.com/datasets/ealaxi/paysim1

[3] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, Mar. 2015, doi: 10.1371/journal.pone.0118432.

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, Article 15, Jul. 2009, doi: 10.1145/1541880.1541882.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 2008 Eighth IEEE Int. Conf. on Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[6] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature," *Decision Support Systems*, vol. 50, no. 3, pp. 559–569, Feb. 2011, doi: 10.1016/j.dss.2010.08.006.